

EE-2003

Computer Organization and Assembly Language

Week 05 – Lecture 10

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

1 Quick Review

2 Example 3.3: Bubble Sort

Quick Review: What is Done in Last Lecture?

- Conditional Jumps
- UnConditional Jumps
- Relative Addressing
- Types of Jump

Bubble Sort Example: Initial Data

Index	1	2	3	4
Data	60	55	45	58

Goal: Sort in ascending order using Bubble Sort.

Bubble Sort: Stepwise Passes and Swap Status

Pass	Comparison	Swap Done	Array State	Swap Flag
1	60 vs 55	Yes	55, 60, 45, 58	On
1	60 vs 45	Yes	55, 45, 60, 58	On
1	60 vs 58	Yes	55, 45, 58, 60	On
2	55 vs 45	Yes	45, 55, 58, 60	On
2	55 vs 58	No	45, 55, 58, 60	Off
2	58 vs 60	No	45, 55, 58, 60	Off
3	45 vs 55	No	45, 55, 58, 60	Off
3	55 vs 58	No	45, 55, 58, 60	Off
3	58 vs 60	No	45, 55, 58, 60	Off

Observation: Sorting stops when a pass completes without any swaps (Swap Flag = Off).

Bubble Sort: Final Sorted Array

Index	1	2	3	4
Data	45	55	58	60

Summary:

- Used Bubble Sort to arrange numbers in ascending order.
- Each pass “bubbled” the largest remaining number to the end.
- Process stopped when a pass occurred with no swaps.

Note: Sorting stops when a pass completes with Swap Flag = Off.

Example 3.3: Bubble Sort (Part 1)

Assembly Code (Initialization)

```
[org 0x0100]
    jmp start

data: dw 60, 55, 45, 50, 40,
      35, 25, 30, 10, 0
swap: db 0

start: mov bx, 0           ; array index
      mov byte [swap], 0 ; reset swap flag
```

- Data array of 10 numbers stored at data.
- swap flag tracks whether a swap occurred.
- BX used as array index.

Example 3.3: Bubble Sort (Part 2)

Main Loop (Comparison + Swap)

```
loop1:  mov ax, [data+bx]      ; load element
        cmp ax, [data+bx+2]   ; compare with next
        jbe noswap            ; no swap if in order

        mov dx, [data+bx+2]   ; swap elements
        mov [data+bx+2], ax
        mov [data+bx], dx
        mov byte [swap], 1    ; swap occurred
noswap:  add bx, 2
        cmp bx, 18             ; last element?
        jne loop1
```

- Comparison is register-to-register (AX vs next element).
- Swap done with AX and DX.
- Loop continues until last element compared.

Example 3.3: Bubble Sort (Part 3)

Termination Condition

```
cmp byte [swap], 1 ; was there a swap?
je  bsort          ; yes -> another pass

mov ax, 0x4c00      ; terminate program
int 0x21
```

- If any swap occurred -> repeat sorting pass.
- If no swap -> list sorted -> terminate.
- JBE vs JB:
 - JBE avoids infinite loops on equal elements.
 - JB would wrongly cause unnecessary swaps.

Example 3.3: Key Notes on Bubble Sort

- BX compared with 18 (not 20) because last comparison is between 9th and 10th element.
- Changing JBE -> JAE makes sorting descending.
- For signed numbers: use JLE / JGE instead of JBE / JAE.
- Example signed data: 60, 55, 45, 50, -40, -35, 25, 30, 10, 0
- JBE (unsigned) treats negative numbers as large positive.
- JLE (signed) treats them correctly as negatives.

Thank You!

Any Questions?

- We explored: **conditional and unconditional jumps, relative addressing, and Bubble Sort.**
- Key lessons:
 - JNE/JNZ share the same opcode.
 - Short jumps use relative offsets, making code efficient and relocatable.
 - Bubble Sort works by repeated passes, stopping when no swaps occur.
 - Choosing the right jump (signed vs unsigned) ensures correct comparisons.
- Takeaway: **Program flow in assembly relies heavily on jumps and flag evaluation.**

“Master jumps and flags — and you master control in Assembly.”